

```
def map[B] (fa: F[A]) (f: A => B): F[B] =
  map2(fa, unit(())) ((a, _) => f(a))
```

Of course, there's something rather arbitrary about this definition—we could have just as easily put the *unit* on the *left* side of the call to `map2`:

```
def map[B] (fa: F[A]) (f: A => B): F[B] =
  map2(unit(()), fa) ((_, a) => f(a))
```

The first two laws for *Applicative* might be summarized by saying that *both* these implementations of `map` respect the functor laws. In other words, `map2` of some `fa: F[A]` with `unit` preserves the structure of `fa`. We'll call these the left and right identity laws (shown here in the first and second lines of code, respectively):

```
map2(unit(()), fa) ((_, a) => a) == fa
map2(fa, unit(())) ((a, _) => a) == fa
```

12.5.2 Associativity

To see the next law, *associativity*, let's look at the signature of `map3`:

```
def map3[A,B,C,D] (fa: F[A],
                  fb: F[B],
                  fc: F[C]) (f: (A, B, C) => D): F[D]
```

We can implement `map3` using `apply` and `unit`, but let's think about how we might define it in terms of `map2`. We have to combine our effects two at a time, and we seem to have two choices—we can combine `fa` and `fb`, and then combine the result with `fc`. Or we could associate the operation the other way, grouping `fb` and `fc` together and combining the result with `fa`. The associativity law for applicative functors tells us that we should get the same result either way. This should remind you of the associativity laws for monoids and monads:

```
op(a, op(b, c)) == op(op(a, b), c)
compose(f, op(g, h)) == compose(compose(f, g), h)
```

The associativity law for applicative functors is the same general idea. If we didn't have this law, we'd need *two* versions of `map3`, perhaps `map3L` and `map3R`, depending on the grouping, and we'd get an explosion of other combinators based on having to distinguish between different groupings.

We can state the associativity law in terms of `product`.⁵ Recall that `product` just combines two effects into a pair, using `map2`:

```
def product[A,B] (fa: F[A], fb: F[B]): F[(A,B)] =
  map2(fa, fb) ((_,_))
```

And if we have pairs nested on the right, we can always turn those into pairs nested on the left:

⁵ `product`, `map`, and `unit` are an alternate formulation of *Applicative*. Can you see how `map2` can be implemented using `product` and `map`?